

Bluetooth Obstacle-Avoiding Rover

Project guide: approach, methods, challenges, and fixes

Author: Sai

This guide explains how I built the Bluetooth Obstacle-Avoiding Rover: a two-mode Arduino Nano robot I can drive from an Android app over Bluetooth, with an autonomous mode and a hard safety override that reverses away from obstacles. It covers my approach, the methods used, and the challenges I solved along the way.

Overview and goal

I wanted a rover that a person could drive manually but that would never let them crash it. So alongside manual driving and a self-navigating autonomous mode, I added a safety override: if an obstacle comes within 50 cm in manual mode, the rover ignores commands, reverses to safety, and waits for a clear path.

How I approached it

I modelled the rover as a state machine with two main modes (manual and autonomous) and a small override sub-state machine layered on top. Everything in the loop is non-blocking, using `millis()` timers rather than `delay()`, so the rover keeps reading commands and the ultrasonic sensor while it moves.

- **Input:** the app sends single ASCII characters over Bluetooth (HC-05).
- **Sensing:** an HC-SR04 ultrasonic sensor measures the distance ahead.
- **Actuation:** an L298N H-bridge drives the two motors, with PWM for speed.
- **Safety:** an override state machine takes priority over user commands.

Methods and techniques used

- Finite state machine for modes and the safety override.
- Non-blocking timing with `millis()` so sensing and control run concurrently.
- Ultrasonic ranging with `pulseIn()`, converting echo time to centimetres.
- Motor control through an L298N: direction pins plus PWM on the enable pins.
- Serial communication over `SoftwareSerial` to the HC-05 Bluetooth module.
- A one-time, time-based reverse-distance calibration to compensate for no encoders.

Implementation walkthrough

Each loop reads any waiting Bluetooth command, refreshes the distance on a timer, then runs the active mode. In manual mode, if the distance drops below the threshold, the override fires: it reverses a fixed distance at a fixed speed, then waits in a stopped state until a fresh, safe command arrives. The reverse uses a constant speed so the distance covered is repeatable no matter how fast I was driving.

```
enum OverridePhase { OV_NONE, OV_BACKING, OV_WAITING_SAFE };  
// distance < 50cm -> reverse a fixed timed distance -> wait for safe command  
backupMs = (BACKUP_DISTANCE_CM / ROVER_REVERSE_SPEED_CMPS) * 1000;
```

Challenges I faced and how I fixed them

- **Only one hardware serial port:** the USB port uses it, so I could not also wire the HC-05 there. **Fix:** I put Bluetooth on `SoftwareSerial`, leaving USB free for uploading and debugging.

- **HC-05 voltage mismatch:** the module's RX is 3.3V-tolerant but the Nano outputs 5V. **Fix:** a resistor voltage divider on the HC-05 RXD line.
- **No wheel encoders to measure distance:** **Fix:** I calibrate the rover's actual reverse speed once, then convert the desired backup distance into a reverse time.
- **Blocking code froze the rover:** using `delay()` made it stop responding. **Fix:** I switched to `millis()`-based timing so sensing and commands run without pausing.
- **Override could be overridden by a held button:** **Fix:** the override is a priority state that locks out movement commands until it completes and a new command arrives.

Testing and what I learned

I tested manual driving first, then walked an obstacle into its path to confirm the reverse-and-lockout, then let it roam in autonomous mode. The project taught me state machine design, why non-blocking timing matters in embedded systems, how H-bridges and PWM control DC motors, and the realities of voltage levels between modules.